

Tarea 4: Componente practico

Curso: Programación

Código: 213023

Leorgy Ailen Baron Balcazar

1.193.133.279

Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI, Universidad Nacional

Abierta y a Distancia

CEAD Santa Marta Magdalena

Docente:

María Alejandra Diaz Mejia

Facultad de Ingenierias

Ingeniería Electronica

Santa Marta Magdalena

11 -05 -2026

Introducción

El desarrollo de sistemas orientados a objetos representa una de las metodologías más utilizadas en la programación moderna debido a su capacidad para organizar procesos complejos de manera estructurada, reutilizable y eficiente. En esta actividad se diseñó un sistema integral para la gestión de clientes, servicios y reservas de la empresa Software FJ, implementando conceptos fundamentales de programación orientada a objetos en Python.

Durante el desarrollo del proyecto se aplicaron principios como abstracción, herencia, encapsulación y polimorfismo, permitiendo construir una aplicación modular capaz de adaptarse a diferentes tipos de servicios y operaciones. Asimismo, se incorporó manejo avanzado de excepciones con el propósito de garantizar estabilidad en la ejecución del programa aun cuando se presenten errores derivados de datos inválidos o procesos incorrectos.

El sistema fue desarrollado sin utilizar bases de datos, empleando únicamente objetos, listas y archivos de registro para el almacenamiento temporal y control de eventos. De esta manera, la actividad permitió fortalecer habilidades relacionadas con el diseño lógico, validación de datos, control de errores y estructuración de aplicaciones orientadas a objetos mediante el lenguaje Python.

Objetivos

Objetivo general

Desarrollar un sistema integral orientado a objetos en Python para la gestión de clientes, servicios y reservas, aplicando principios de programación orientada a objetos y manejo avanzado de excepciones.

Objetivos específicos

- Implementar clases abstractas y clases derivadas para representar las entidades principales del sistema.
- Aplicar encapsulación y validación de datos en el registro de clientes.
- Utilizar herencia y polimorfismo para administrar distintos tipos de servicios ofrecidos por la empresa.
- Diseñar un sistema de reservas capaz de confirmar, cancelar y procesar operaciones de manera controlada.
- Incorporar manejo avanzado de excepciones mediante estructuras try/except y excepciones personalizadas.
- Registrar errores y eventos importantes en archivos logs para mejorar el control del sistema.
- Simular operaciones válidas e inválidas que permitan verificar la estabilidad y funcionamiento del programa.

Código: <https://github.com/Leorgy908/SistemaReservasPython>

El programa desarrollado corresponde a un sistema orientado a objetos para administrar clientes, servicios y reservas de una empresa llamada Software FJ. El sistema fue construido en Python aplicando principios de programación orientada a objetos y manejo avanzado de excepciones.

Importación de librerías

El programa inicia importando las librerías necesarias para el funcionamiento del sistema. La librería abc fue utilizada para crear clases abstractas y métodos abstractos, mientras que logging permitió registrar errores y eventos importantes dentro de un archivo externo.

La utilización de estas herramientas facilitó la construcción de un sistema más organizado y preparado para manejar errores de manera controlada.

Configuración del archivo de logs

Posteriormente se configuró un archivo llamado eventos.log, encargado de almacenar automáticamente los errores detectados durante la ejecución del programa. Este registro permite mantener un historial de fallos y facilita la supervisión del funcionamiento del sistema sin detener la ejecución de la aplicación.

Creación de excepciones personalizadas

Con el fin de mejorar el control de errores se implementaron excepciones personalizadas para clientes, servicios y reservas. Estas excepciones permitieron identificar de manera más precisa el origen de los problemas detectados durante la ejecución.

Gracias a esta estrategia, el sistema pudo manejar errores específicos sin afectar el funcionamiento general de la aplicación.

Clase abstracta Persona

Se creó una clase abstracta denominada Persona, utilizada como base para representar entidades generales dentro del sistema. Esta clase contiene atributos básicos como nombre y documento, además de un método abstracto encargado de mostrar información de cada persona registrada.

La abstracción permitió establecer una estructura común que posteriormente fue heredada por otras clases del sistema.

Clase Cliente

La clase Cliente hereda de la clase abstracta Persona e incorpora validaciones relacionadas con el nombre, documento y correo electrónico. Durante el desarrollo se implementó encapsulación utilizando atributos privados para proteger la información del correo electrónico del cliente.

Además, se utilizaron métodos getter y setter para consultar y modificar la información de manera segura. Estas validaciones ayudaron a evitar el registro de datos inconsistentes o incorrectos.

Clase abstracta Servicio

Se diseñó una clase abstracta llamada Servicio, encargada de representar los diferentes servicios ofrecidos por la empresa. Esta clase define atributos comunes como nombre y tarifa base, además de métodos abstractos para calcular costos y mostrar descripciones.

La implementación de esta clase permitió construir diferentes servicios especializados mediante herencia y polimorfismo.

Servicios especializados

A partir de la clase Servicio se desarrollaron tres servicios especializados:

- Reserva de salas.
- Alquiler de equipos.

- Asesorías especializadas.

Cada servicio implementó sus propios métodos de cálculo de costos y descripción, demostrando el uso de polimorfismo dentro del sistema. Además, algunos servicios incorporaron descuentos, impuestos y recargos opcionales, permitiendo mayor flexibilidad en las operaciones realizadas.

Clase Reserva

La clase Reserva fue diseñada para relacionar clientes, servicios y duración de las reservas. Dentro de esta clase se implementaron funciones para confirmar, cancelar y procesar reservas.

Durante el procesamiento se utilizaron estructuras try/except para controlar errores derivados de operaciones inválidas. Cuando se detectaba un problema, el sistema registraba automáticamente el error en el archivo logs y continuaba ejecutándose sin interrupciones.

Manejo de listas y almacenamiento temporal

Debido a que la actividad no permitía utilizar bases de datos, se implementaron listas para almacenar clientes, servicios y reservas temporalmente. Esta estrategia permitió gestionar la información de manera sencilla y mantener la organización interna del sistema.

Simulación de operaciones

El sistema ejecutó diferentes operaciones válidas e inválidas con el propósito de comprobar la robustez de la aplicación. Se realizaron registros correctos de clientes, creación de servicios y reservas exitosas, así como intentos inválidos que generaron excepciones controladas.

Estas pruebas permitieron demostrar que el programa podía continuar funcionando aun cuando se presentaran errores durante la ejecución.

Manejo avanzado de excepciones

Finalmente, se incorporaron estructuras avanzadas de manejo de errores utilizando bloques try, except, else y finally. Esto permitió controlar entradas inválidas del usuario y garantizar que determinadas acciones se ejecutaran independientemente de la presencia de errores.

La implementación de estas estructuras fortaleció la estabilidad y confiabilidad del sistema desarrollado.

Conclusión

El desarrollo de esta actividad permitió comprender la importancia de la programación orientada a objetos en la construcción de aplicaciones organizadas, reutilizables y fáciles de mantener. La implementación de clases abstractas, herencia y polimorfismo facilitó la estructuración del sistema y permitió representar de manera eficiente los diferentes servicios y procesos involucrados en la gestión de reservas.

Asimismo, el manejo avanzado de excepciones contribuyó significativamente a la estabilidad de la aplicación, permitiendo controlar errores sin detener la ejecución del programa. El uso de validaciones y registros en archivos logs ayudó a mejorar la confiabilidad del sistema y el seguimiento de eventos importantes durante su funcionamiento.

Finalmente, la práctica fortaleció las habilidades de programación en Python y permitió aplicar conceptos teóricos en un entorno práctico, evidenciando la utilidad de la programación orientada a objetos en el desarrollo de soluciones informáticas robustas y escalables.

Referencias bibliográficas

- Deitel, P., & Deitel, H. (2019). *Python for programmers*. Pearson Education.
- Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos*. McGraw-Hill.
- Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
- Python Software Foundation. (2024). *Python documentation*. [Python Official Documentation](https://docs.python.org/3/)
- Schildt, H. (2018). *Python: The complete reference*. McGraw-Hill Education.
- Sommerville, I. (2016). *Ingeniería de software* (10.^a ed.). Pearson Educación.
- Universidad Nacional Abierta y a Distancia. (2025). *Guía de actividades y rúbrica de evaluación – Curso Programación 213023*. UNAD.
- Zelle, J. (2017). *Python programming: An introduction to computer science*. Franklin, Beedle & Associates.